

# Cas d'exception & contrôle des entrées

Toutes les entrées de **Petit Rayon de Soleil** sont validées **côté serveur**, jamais uniquement côté client. Chaque cas d'erreur renvoie un code HTTP explicite et un message clair, plutôt qu'un plantage.

---

**Projet** Petit Rayon de Soleil    **Auteur** de Breyne Hélió    **Principe** ne jamais faire confiance à l'entrée

**Pourquoi côté serveur ?** Les contrôles côté navigateur (attribut `required`, etc.) améliorent le confort mais peuvent être contournés (requête directe à l'API, client modifié). La **seule validation qui protège réellement** les données est celle du serveur. C'est celle documentée ici.

## 1 Les sept cas d'exception

Chaque ligne : la situation, le contrôle qui la détecte dans le code, le code HTTP renvoyé et le message. Comportements vérifiés par appels réels à l'API.

| Cas d'exception                 | Contrôle serveur                                      | Réponse | Message renvoyé                                  |
|---------------------------------|-------------------------------------------------------|---------|--------------------------------------------------|
| <b>Message vide</b>             | <code>!content    content.trim().length &lt; 5</code> | 400     | « Le message doit faire au moins 5 caractères. » |
| <b>Message trop court</b>       | <code>content.trim().length &lt; 5</code>             | 400     | « Le message doit faire au moins 5 caractères. » |
| <b>Utilisateur non connecté</b> | <code>requireAuth (token absent/invalid)</code>       | 401     | « Non authentifié »                              |
| <b>Utilisateur non admin</b>    | <code>requireAdmin (role !== 'admin')</code>          | 403     | « Accès réservé à l'administrateur »             |
| <b>Favori déjà existant</b>     | <code>INSERT OR IGNORE (idempotent)</code>            | 201     | Ignoré silencieusement, pas de doublon créé      |
| <b>Identifiants invalides</b>   | <code>!user    !checkPassword(...)</code>             | 401     | « Email ou mot de passe incorrect. »             |
| <b>Erreur base de données</b>   | <code>wrap() → middleware d'erreurs global</code>     | 500     | « Erreur interne du serveur. »                   |

**PRÉCISION** Les contrôles s'appliquent dans l'ordre du traitement. Ainsi, un message vide envoyé *sans être connecté* renvoie d'abord 401 (l'authentification est vérifiée avant le contenu). Une fois connecté, le même message vide renvoie bien 400.

## 2 Contrôle des entrées (exemples de code)

Chaque route valide ses entrées avant tout traitement. Extraits réels de `server.js`.

### Proposition de message — longueur contrôlée

`server.js` — POST `/api/messages`

```
const { content } = req.body || {};  
if (!content || content.trim().length < 5) {  
  return res.status(400).json({ error: 'Le message doit faire au moins 5 caractères.' });  
}
```

### Inscription — champs requis & email unique

`server.js` — POST `/api/register`

```
const { name, email, password } = req.body || {};  
if (!name || !email || !password) {  
  return res.status(400).json({ error: 'Nom, email et mot de passe sont requis.' });  
}  
if (password.length < 6) {  
  return res.status(400).json({ error: 'Le mot de passe doit faire au moins 6 caractères.' });  
}  
// Email déjà pris → 409 Conflict  
const existing = await db.prepare('SELECT id FROM users WHERE email = ?').get(email);  
if (existing) return res.status(409).json({ error: 'Cet email est déjà utilisé.' });
```

### Favori en double — INSERT OR IGNORE

Plutôt que de renvoyer une erreur si le favori existe déjà, la requête l'ignore : l'opération est **idempotente** (la rejouer ne change rien). L'utilisateur n'a pas à se soucier de savoir s'il a déjà mis ce message en favori.

`server.js` — POST `/api/favoris`

```
// INSERT OR IGNORE : pas d'erreur si le favori existe déjà.  
await db.prepare('INSERT OR IGNORE INTO favoris (user_id, message_id) VALUES (?, ?)')  
  .run(req.user.id, message_id);
```

### 3 Filet de sécurité : erreurs imprévues

Au-delà des validations explicites, toute exception non prévue (erreur SQL, bug) est rattrapée pour éviter de laisser une requête sans réponse ou de faire tomber le serveur.

Deux mécanismes se combinent : un **wrapper** qui capture les erreurs des fonctions asynchrones, et un **middleware d'erreurs global** qui renvoie une réponse 500 propre.

server.js – capture + middleware d'erreurs

```
// Enveloppe chaque handler async : redirige toute erreur vers le middleware.
const wrap = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);

// Middleware d'erreurs global (4 paramètres) : dernier rempart.
app.use((err, req, res, _next) => {
  console.error('Erreur serveur :', err);
  res.status(500).json({ error: 'Erreur interne du serveur.' });
});
```

#### Preuve : une erreur SQL ne fait pas planter le serveur

Test réel provoquant une requête sur une table inexistante :

test d'une erreur base de données

```
$ curl /boom # route qui déclenche une erreur SQL volontaire
[handler] erreur interceptée : SQLITE_ERROR: no such table: ...
erreur base de données → 500 { "error": "Erreur interne du serveur." }
(le process ne crashe pas, une réponse 500 propre est renvoyée)
```

**Message d'erreur volontairement générique.** Le 500 ne révèle jamais le détail technique (nom de table, requête SQL) au client : ces informations restent dans les logs serveur. Exposer une trace d'erreur serait une fuite d'information exploitable.

## 4 Ce que garantit ce dispositif

---

- **Aucune entrée n'est prise pour argent comptant** : présence, longueur et validité sont vérifiées côté serveur.
- **Chaque erreur a un code HTTP juste** : 400 (entrée invalide), 401 (non authentifié), 403 (interdit), 404 (introuvable), 409 (conflit), 500 (erreur interne).
- **Les accès sont contrôlés avant l'action** : les middlewares `requireAuth` / `requireAdmin` s'exécutent en amont des handlers.
- **Aucune exception ne fait tomber le serveur** : le wrapper + le middleware global forment un dernier rempart.
- **Les injections SQL sont neutralisées** : toutes les requêtes sont paramétrées ( `?` ), l'entrée est traitée comme une donnée, jamais comme du code.

---

Petit Rayon de Soleil — Annexe « Cas d'exception & contrôle des entrées ». Codes de retour vérifiés par appels réels à l'API (message vide/court, identifiants invalides, non connecté, non admin, erreur base).